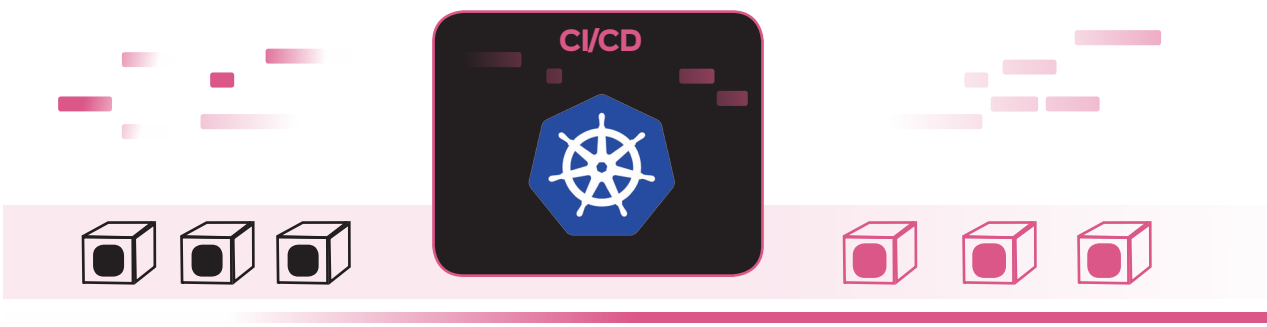
Setting Up a CI/CD Pipeline with Kubernetes

A continuous integration/continuous deployment (CI/CD) pipeline is the spine of the modern DevOps environment. It bridges the gap between the development and operations teams by automating the building, testing and deployment of applications. This article tells you how to set up a CI/CD pipeline using Kubernetes.



Jenkins is an open source/free continuous integration and continuous delivery tool, which can be used to automate the building, testing and deployment of software. It is generally considered the most accepted automation server, and is used by more than a million users worldwide. Jenkins is the best choice for implementing CI/CD.

In this article, we will first try to understand what a CI/CD pipeline is and why it is important. We will then try to set up a CI/CD pipeline with the help of Kubernetes. So, let's start.

What is a pipeline and what is CI/CD?

In computer science, a pipeline can also be called a data pipeline. It is a set of data processing elements connected in series, where the output of one element is the input of the next one. The components of a pipeline are often executed in parallel or in a time-sliced fashion. While CI stands for continuous integration, CD stands for continuous delivery/continuous deployment. Evidently, continuous integration is a set of exercises that makes development teams implement small changes and check code to version control repositories regularly. The main goal of CI is to create a consistent and automated way to build, package and test applications. Continuous delivery picks up where continuous integration ends. CD automates the delivery of applications to particular infrastructure environments. Many teams work with numerous environments other than production, such as development and QA environments, and CD makes sure there is an automated way to push code changes to them.

Why use it?

The CI/CD pipeline focuses resources on things that matter by automating the process and delegating it to a CI/CD pipeline. Resources are freed for actual product development tasks and the chance of errors is reduced.

Increase transparency and visibility: When a CI/CD pipeline is set up, the entire team knows what's going on with the build as well as gets the latest results of tests, which means the team can raise issues and plan its work in context.

Detect and fix issues early: CI/CD pipeline deployment is automated and fast, which means the tester/QA gets more time to detect problems and developers get more time to fix them. It can be built and deployed any number of times without any effort. Hence, software becomes more bug-free.

Improve quality and testability: Easier testing makes it easier to achieve quality. Testability has many dimensions —it can be considered by how observable, controllable and decomposable the outcome is. Testability is affected by how effortlessly new builds are accessible and what tools are used. Continuous integration and delivery writes tests, runs them, and also delivers builds regularly and consistently.

The prerequisites for setting up a CI/CD pipeline are:

1. Docker engine should be installed on the platform.
2. *minikube* and *kubectl* should be installed on the platform.